

Nous – Desktopowa platforma do przeprowadzania testów psychometrycznych z opcjonalną synchronizacją wyników do chmury

Bartosz Kauc

2026-03-26

Spis treści

1	Zakładany Plan Pracy	1
1.1	Temat pracy inżynierskiej	1
1.2	Cel i zakres projektu	1
1.3	Stos technologiczny	2
1.4	Etapy realizacji pracy	2
1.4.1	Etap 1 — Analiza wymagań i projektowanie architektury	2
1.4.2	Etap 2 — Implementacja jądra aplikacji (proces główny)	2
1.4.3	Etap 3 — Implementacja interfejsu użytkownika i modułów frontendu	3
1.4.4	Etap 4 — Bezpieczeństwo danych wynikowych	3
1.4.5	Etap 5 — Tryb wysokiej precyzji (HPM)	4
1.4.6	Etap 6 — Wersja przeglądarkowa	4
1.4.7	Etap 7 — Strona informacyjna projektu	5
1.4.8	Etap 8 — Testowanie	5
1.4.9	Etap 9 — CI/CD i publikacja	5
1.5	Struktura katalogów projektu	6
1.6	Spodziewane wyniki	6
1.7	Wnioski i praktyczne wykorzystanie	7
2	Oswiadczenie odnośnie wykorzystania GenAI przy tworzeniu pracy	7

1 Zakładany Plan Pracy

1.1 Temat pracy inżynierskiej

Nous – Desktopowa platforma do przeprowadzania testów psychometrycznych z opcjonalną synchronizacją wyników do chmury

1.2 Cel i zakres projektu

Celem projektu było zaprojektowanie i implementacja wieloplatformowej aplikacji desktopowej służącej do zarządzania i uruchamiania testów psychometrycznych. Aplikacja dostarcza środowisko, w którym niezależnie opracowane testy można instalować, uruchamiać, a ich wyniki — zbierać, przechowywać lokalnie i synchronizować z bazą danych w chmurze.

Projekt zrealizował następujące cele:

- Stworzenie wyizolowanego, bezpiecznego środowiska uruchomieniowego dla testów (zarówno webowych, jak i natywnych Python/PsychoPy).
- Umożliwienie badaczom zarządzania zestawem testów z poziomu jednego interfejsu graficznego.
- Zaimplementowanie szyfrowania i integralności wyników badań.
- Zapewnienie synchronizacji wyników z bazą danych Firebase (Firestore).
- Opublikowanie aplikacji jako natywnego pakietu instalacyjnego na systemy Windows, macOS i Linux oraz wersji przeglądarkowej.

1.3 Stos technologiczny

Warstwa	Technologia
Aplikacja desktopowa	Electron v40 (Node.js + Chromium)
Język frontendu	JavaScript (ES Modules)
Baza danych chmurowa	Firebase Firestore
Uwierzytelnianie	Firebase Authentication
Lokalna baza danych	IndexedDB (przez bibliotekę <code>idb</code>)
Szyfrowanie	AES-GCM 256-bit (Web Crypto API) + HMAC-SHA256 (<code>crypto</code> Node.js) + PBKDF2 (KDF z PIN)
Zarządzanie kluczami	Electron <code>safeStorage</code>
Tryb HPM	Python / PsychoPy (osadzony interpreter)
Testowanie jednostkowe	Vitest + happy-dom + fake-indexeddb
Testowanie E2E	Playwright
CI/CD	GitHub Actions
Strona projektu	Jekyll (GitHub Pages)
Pakowanie aplikacji	electron-builder (NSIS dla Windows, DMG dla macOS, AppImage dla Linux)

1.4 Etapy realizacji pracy

1.4.1 Etap 1 — Analiza wymagań i projektowanie architektury

Na pierwszym etapie przeprowadzona została analiza wymagań funkcjonalnych i нефункциональных systemu. Zdefiniowane zostały przypadki użycia:

- badacz pobiera test z biblioteki chmurowej i go uruchamia,
- uczestnik wypełnia ankietę demograficzną przed badaniem,
- wyniki badania są zapisywane lokalnie, podpisywane i opcjonalnie synchronizowane z Firestore.

Podjęta została decyzja o wyborze Electrona jako frameworka, uzasadniona koniecznością dostępu do systemu plików (zapis wyników, instalacja testów), możliwością integracji z interpreterem Python oraz wsparciem dla wielu platform. Zaprojektowana została modułowa architektura aplikacji z wyraźnym podziałem na procesy: główny (`main.js`) oraz renderer (`src/`).

Zdefiniowane zostały wymagania bezpieczeństwa: szyfrowanie wyników, walidacja źródeł pobieranych paczek ZIP, ochrona przed atakami ZIP Slip.

1.4.2 Etap 2 — Implementacja jądra aplikacji (proces główny)

Zaimplementowany został plik `main.js` odpowiedzialny za zarządzanie cyklem życia aplikacji Electron oraz komunikację IPC między procesem głównym a rendererem.

Zrealizowane funkcjonalności:

- **Tworzenie okna głównego:** Konfiguracja `BrowserWindow` z włączoną izolacją kontekstu i wyłączoną integracją `Node.js` w rendererze (bezpieczeństwo).
- **Pobieranie testów:** Obsługa kanału IPC `download-and-run`, implementacja kolejki pobierania (`downloadQueue`), obsługa przekierowań HTTP, rozpakowanie ZIP z walidacją ścieżek (ochrona przed ZIP Slip), zapis `meta.json`.
- **Uruchamianie testów JS:** Otwieranie testu w osobnym oknie pełnoekranowym (`openTestWindow`) z dedykowanym `preload_test.js`.
- **Uruchamianie testów Python (HPM):** Spawning procesu Pythona z osadzonego środowiska (`runPythonTestIfPossible`), obsługa wyników przez IPC.
- **Zarządzanie kluczem szyfrowania:** Generowanie i przechowywanie 256-bitowego klucza AES przy użyciu `safeStorage` Electrona; klucz nie jest nigdy przechowywany w postaci jawnej.
- **Podpisywanie HMAC:** Każdy zapisany wynik badania został podpisany kluczem HMAC-SHA256 (wygenerowanym z klucza głównego), co umożliwia weryfikację integralności danych.
- **Lokalny zapis wyników:** Wyniki zapisywane są do pliku `.json` w folderze danych użytkownika (`app.getPath('userData')`).
- **Skanowanie biblioteki lokalnej:** Kanał IPC `get-local-versions` skanuje folder `tests_library`, odczytuje wersje z `meta.json` i informuje renderer o stanie instalacji.
- **Aktualizacja aplikacji:** Integracja `electron-updater` pobierającego nowe wersje launchera z GitHub Releases.

Zapewniona została kompatybilność ze wszystkimi obsługiwanymi platformami (Windows, macOS, Linux), w tym obsługa różnych ścieżek danych na każdym z systemów.

1.4.3 Etap 3 — Implementacja interfejsu użytkownika i modułów frontendu

Cała logika warstwy prezentacji została podzielona na moduły ES importowane przez główny punkt wejścia `src/app.js`. Aplikacja dostarcza użytkownikowi następujące możliwości:

- **Przeglądanie i zarządzanie biblioteką testów** — wyszukiwanie, filtrowanie i tagowanie testów synchronizowanych z chmurą w czterech trybach widoku.
- **Uruchamianie testów** — uruchamianie testów webowych i Python w izolowanym środowisku pełnoekranowym.
- **Zbieranie danych demograficznych** — konfigurowalny kreator ankiet demograficznych wypełnianych przed każdym badaniem.
- **Historia wyników** — przeglądanie, filtrowanie i eksport wyników do CSV z automatycznym deszyfrowaniem wyników chmurowych.
- **Synchronizacja z chmurą** — szyfrowane przesyłanie wyników do Firestore z obsługą trybu offline.
- **Zarządzanie szyfrowaniem E2E** — interfejs do tworzenia i zarządzania kluczem End-to-End chronionym PIN-em użytkownika.
- **Ustawienia aplikacji** — personalizacja motywu, koloru akcentu i rozmiaru czcionki.
- **Aktualizacje** — informowanie użytkownika o nowych wersjach launchera i możliwość ich instalacji.

Szczegółowy opis implementacji poszczególnych modułów zawarty jest w rozdziale poświęconym implementacji.

1.4.4 Etap 4 — Bezpieczeństwo danych wynikowych

Realizacja wymagań bezpieczeństwa jest kluczowym elementem systemu, gdyż aplikacja operuje na danych z badań naukowych.

Zrealizowane mechanizmy:

- **Szyfrowanie wyników (lokalne)** — dane wynikowe w IndexedDB szyfrowane są algorytmem AES-GCM kluczem 256-bitowym zarządzanym przez `safeStorage`. Każdy rekord posiada unikalne IV (Initialization Vector).
 - **Szyfrowanie E2E w modelu Zero-Knowledge** — klucz chmurowy AES-GCM 256-bit generowany losowo, owijany (wrapped) kluczem pochodnym z 6-cyfrowego PIN-u użytkownika (PBKDF2, 200 000 iteracji, SHA-256). Serwer Firebase przechowuje wyłącznie zaszyfrowaną wersję klucza — nigdy nie ma dostępu do danych w postaci jawnej. Klucz jest przechowywany lokalnie w Electron `safeStorage` po pierwszym odszyfrowaniu.
 - **Integralność plików lokalnych** — każdy wynik zapisywany na dysk jest podpisany HMAC-SHA256. Umożliwia to weryfikację, czy plik nie był modyfikowany od chwili zapisu.
 - **Walidacja pobieranych paczek** — przed rozpakowaniem paczki ZIP sprawdzane są ścieżki wszystkich wpisów w celu wykrycia ataków ZIP Slip. URL musi spełniać jednocześnie dwa warunki: (1) protokół HTTPS, (2) hostname to `github.com` lub `raw.githubusercontent.com` i ścieżka zaczyna się od `/KaucBartosz/`. Jedynym wyjątkiem jest `objects.githubusercontent.com` (CDN GitHub Releases), który używa haszowanych URL niezawierających nazwy właściciela. Dzięki temu uniemożliwione jest wskazanie jako źródło zasobów spoza repozytorium autora.
 - **Walidacja ID testów** — identyfikatory testów walidowane są wyrażeniem regularnym `[a-zA-Z0-9_-]+` przed użyciem jako ścieżka systemu plików.
 - **Izolacja kontekstu** — okna testów uruchamiane są z wyłączoną integracją Node.js i włączoną izolacją kontekstu; komunikacja wyłącznie przez preload API.
-

1.4.5 Etap 5 — Tryb wysokiej precyzji (HPM)

Wiele testów psychometrycznych wymaga dokładności pomiaru czasu rzędu pojedynczych milisekund, której web renderer nie jest w stanie zapewnić. Aby temu zaradzić, zdefiniowany i zaimplementowany został Tryb Wysokiej Precyzji (HPM — High Precision Mode). Pozwolił on również na uruchamianie testów napisanych w języku Python, co umożliwia korzystanie z zewnętrznej aparatury badawczej.

Zrealizowane elementy:

- Pobieranie i instalacja osadzonego środowiska Python (ok. 300 MB) jako jednorazowej operacji.
 - Wykrycie obecności pliku `main.py` w folderze testu.
 - Uruchamianie testu jako procesu potomnego Pythona (`spawn`) zamiast okna przeglądarki.
 - Obsługa wyników przez lokalny socket IPC lub stdout.
 - Obsługa specyfiki poszczególnych dystrybucji Linux (detekcja Debian/Ubuntu vs. Fedora/RHEL i informowanie użytkownika o wymaganych zależnościach systemowych).
 - Sprawdzanie i aktualizacja silnika HPM przez Firestore.
-

1.4.6 Etap 6 — Wersja przeglądarkowa

Aplikacja została przygotowana do uruchomienia bezpośrednio w przeglądarce internetowej (bez instalacji) z zachowaniem możliwie dużej funkcjonalności.

Ograniczenia wersji przeglądarkowej:

- Brak dostępu do systemu plików — wyniki eksportowane natychmiast jako plik CSV.
 - Brak widoku historii i aktualizacji (nie dotyczy wersji web).
 - Brak synchronizacji i opcji wylogowania.
 - Brak trybu HPM.
 - Automatyczne logowanie jako gość bez ekranu logowania.
 - Blokada urządzeń dotykowych (tablety/smartfony) — platforma wymaga klawiatury i myszy.
-

1.4.7 Etap 7 — Strona informacyjna projektu

Stworzona została strona informacyjna projektu hostowana na GitHub Pages, zbudowana z użyciem generatora stron statycznych Jekyll.

Zawartość strony:

- Strona główna z opisem projektu i przyciskiem pobierania najnowszej wersji.
- Strona funkcjonalności z opisem możliwości platformy.
- Automatyczne osadzanie numeru najnowszej wersji i linków do plików instalacyjnych przez GitHub API.

1.4.8 Etap 8 — Testowanie

Weryfikacja poprawności implementacji została przeprowadzona na dwóch poziomach.

1.4.8.1 Testy jednostkowe (Vitest) Testy objęły następujące moduły:

Plik testowy	Testowany moduł
<code>utils.test.js</code>	<code>debounce</code> , <code>escapeHtml</code> , <code>flattenObject</code> , <code>sortByInstallStatus</code>
<code>cryptoService.test.js</code>	Szyfrowanie i deszyfrowanie AES-GCM
<code>database.test.js</code>	CRUD na IndexedDB (<code>fake-indexeddb</code>)
<code>dialog.test.js</code>	Tworzenie i zamykanie okienek dialogowych
<code>library.test.js</code>	Renderowanie kart testów, filtrowanie, sortowanie
<code>results.test.js</code>	Obsługa wyników i zapis do bazy
<code>settings.test.js</code>	Zastosowanie ustawień motywu i koloru
<code>ui.test.js</code>	Przełączanie widoków
<code>whatsNew.test.js</code>	Ładowanie widoku nowości

Środowisko testowe: Vitest z `happy-dom` (emulacja DOM) i `fake-indexeddb` (emulacja IndexedDB). Raportowanie pokrycia kodu przez `@vitest/coverage-v8`.

1.4.8.2 Testy End-to-End (Playwright) Testy E2E uruchomiły rzeczywistą aplikację Electron i symulowały działania użytkownika: logowanie, nawigację, pobieranie testu, wyświetlanie wyników.

1.4.9 Etap 9 — CI/CD i publikacja

Wdrożony został potok GitHub Actions realizujący następujące zadania:

- **release.yml** — Automatyczne budowanie instalatorów (`.exe` dla Windows, `.AppImage` dla Linux, `.dmg` dla macOS) po opublikowaniu nowego Release na GitHub i wgranie ich jako artefaktów wydania, co umożliwia automatyczne aktualizacje przez `electron-updater`.
- **update-web-tests.yml** — Automatyczne pobieranie aktualnych wersji testów z ich repozytoriów i commit do gałęzi, zapewnienie aktualności wersji przeglądarkowej.
- **hpm-packs.yml** — Budowanie i publikowanie paczek silnika HPM.

Konfiguracja `electron-builder` zapewniła:

- Tworzenie instalatora NSIS (Windows) z możliwością wyboru katalogu instalacji.
- Tworzenie pakietu DMG (macOS).
- Tworzenie paczki AppImage (Linux).
- Różnicowe pakiety aktualizacji (`differentialPackage`).

1.5 Struktura katalogów projektu

```
Nous/
|-- main.js           # Proces główny Electron (IPC, pobieranie, HPM, szyfrowanie)
|-- preload.js        # API wystawione rendererowi okna głównego
|-- preload_test.js   # API wystawione oknu testowemu
|-- index.html        # Główny plik HTML aplikacji
|-- style.css         # Globalny arkusz stylów
|-- src/
|   |-- app.js        # Punkt wejścia frontendu, inicjalizacja modułów
|   |-- firebaseConfig.js # Konfiguracja Firebase SDK
|   +-- modules/      # Moduły biznesowe
|       |-- appUpdater.js
|       |-- auth.js
|       |-- cryptoService.js
|       |-- e2e.js
|       |-- database.js
|       |-- demoCreator.js
|       |-- demographics.js
|       |-- dialog.js
|       |-- history.js
|       |-- library.js
|       |-- recaptcha.js
|       |-- results.js
|       |-- settings.js
|       |-- sync.js
|       |-- tags.js
|       |-- ui.js
|       |-- updates.js
|       |-- utils.js
|       +-- whatsNew.js
|-- tests/
|   |-- unit/         # Testy jednostkowe Vitest
|   +-- e2e/          # Testy E2E Playwright
|-- docs/             # Strona GitHub Pages (Jekyll)
|-- .github/workflows/ # Potoki CI/CD
+-- package.json      # Konfiguracja projektu i electron-builder
```

1.6 Spodziewane wyniki

Po realizacji wszystkich etapów system spełnia następujące wymagania:

- Aplikacja instaluje się i działa na systemach Windows, macOS i Linux.
 - Badacz może przeglądać i zarządzać biblioteką testów synchronizowaną z chmurą.
 - Każdy test uruchamiany jest w izolowanym, pełnoekranowym środowisku.
 - Wyniki badania są szyfrowane i podpisane kryptograficznie, zapewniając poufność i integralność danych.
 - Wersja przeglądarkowa umożliwia przeprowadzenie badania bez instalacji aplikacji.
 - Automatyczny system aktualizacji informuje użytkownika o nowych wersjach launchera i testów.
 - Pokrycie kodu testami jednostkowymi jest mierzalne i raportowane przy każdym push.
-

1.7 Wnioski i praktyczne wykorzystanie

Aplikacja Nous została oddana do praktycznego użytku. Poniżej zestawiono dotychczasowe przypadki jej zastosowania:

- **21.03.2026** — Wykorzystanie programu przy badaniach psychometrycznych prowadzonych przez „Studenckie Koło Naukowe Psychologii Transportu”.

Planowane są kolejne współprace z jednostkami naukowymi i kołami studenckimi, co pozwoli na dalszą weryfikację przydatności platformy w środowisku akademickim.

2 Oświadczenie odnośnie wykorzystania GenAI przy tworzeniu pracy

Użycie GenAI zostało ograniczone do roli wspierającej zamiast roli wiodącej. Użycie GenAI w takich miejscach jak:

- Logika działania
- Kod programu (z wyjątkami wyjaśnionymi poniżej)
- Pomysł na pracę

Zostało ograniczone do roli konsultanta. Oznacza to, że GenAI wspierało proces szukania rozwiązań, sposobów implementacji i szukania błędów. Kod programu był tworzony ręcznie przy inspiracji z istniejących ogólnodostępnych źródeł, rozwiązań i technik.

Gdzie GenAI było wykorzystane:

- Wygenerowanie plików graficznych: logo.png
- Stylizacja: pliki .html oraz .css
- Tworzenie testów jednostkowych

Uzasadnienie użycia:

- Posłużyłem się GenAI w celu zapewnienia jak najlepszej jakości warstwy wizualnej aplikacji oraz przy tworzeniu testów jednostkowych w celu utrwalenia już osiągniętej funkcjonalności programu.